

# SemantOS: Toward Auditable, Knowledge-Grounded Kernel Tuning with Bundle-Level Risk Gating

Anonymous submission

## Abstract

Language models are increasingly proposed as kernel and database tuners, yet their value over an equally informed optimizer, and the safety of applying their proposals, are rarely measured with independent outcomes. We present SemantOS, a control loop that separates evidence (a typed interaction graph and retrieved traces), proposals (a retrieval-augmented reasoner emitting multi-knob bundles), and enforcement (a runtime that validates, vetoes or stages a complete bundle). We make three contributions. First, we specify a bundle-level control contract and implement it as a dry-run REST prototype together with a separate bounded per-thread actuator. Second, we give an expected-cost identity for gated acceptance and a class-conditional rank gate whose conditioning and exchangeability assumptions are explicit. Third, we define a role-separated evaluation protocol with raw-log verification and apply it to Linux timer-slack/affinity interventions (8,000 grid events, 80 staged executions). Equally informed non-LLM selectors both choose the default configuration, so held-out replay shows no improvement; a frozen gate accepts 5 of 25 and 4 of 24 unsafe runs under a period shift, exceeding its nominal level; and all 80 staged executions restore both original settings. These results show that the protocol exposes unfavorable outcomes rather than hiding them. The LLM’s incremental benefit and explanation faithfulness remain open: the released audit tools are ready, but no model inference was available in our measurement environment.

**Keywords.** Kernel tuning; knowledge representation; selective prediction; provenance; experimental methodology.

## Introduction

Kernel tuning requires choosing configurations whose effects depend on workload, hardware and the current operating state, and a setting that improves one metric can worsen another. Learned tuners have long been applied to databases and data-processing systems (Van Aken et al. 2017; Zhang et al. 2019; Herodotou, Chen, and Lu 2020), and language models now read manuals, propose complete configurations, or search kernel build options (Trummer 2022; Lao et al. 2024; Giannakouris and Trummer 2025; Chen et al. 2024;

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Lin et al. 2025). Two questions are nevertheless hard to answer from existing evaluations. Does a language model add value beyond the information already present in its knowledge base (KB) when a conventional optimizer receives the same inputs? And when a proposal is applied, is the risk of harm measured by an outcome that does not depend on the enforcement mechanism itself?

SemantOS is designed to make both questions testable. It separates three responsibilities: evidence acquisition, candidate generation and enforcement. A typed graph and retrieved traces expose the available evidence; a reasoner proposes a multi-knob bundle; and a runtime checks the complete bundle and decides whether to veto it or enter a staged rollout. Because the evidence is explicit, the same evidence can be handed to a non-LLM optimizer; because enforcement is independent, its errors can be counted against labels defined from raw observations.

**Contributions.** (i) A bundle-level control contract, with a dry-run REST implementation and a separate actuator that applies, verifies and restores per-thread kernel settings (see SemantOS Design). (ii) An expected-cost identity for gated acceptance and a restricted rank gate, stated with the conditioning and assumptions under which they hold (see Probability Statements for Gated Acceptance). (iii) A role-separated evaluation protocol that freezes selections and thresholds before test data are opened and reports every denominator, applied to real Linux interventions (see Measurements and Evaluation Protocol). The measured results are deliberately reported even where unfavorable: informed non-LLM selectors yield no gain over the default, and the frozen gate misses more unsafe runs than its nominal level under a period shift.

**Scope.** The REST runtime never writes kernel settings, and the actuator changes only an owned child thread. We therefore make no deployment-efficacy, production-safety or LLM-superiority claim; the Evaluation Protocol section lists the matched comparisons that such claims require.

## Background and Related Work

**Configuration tuning for systems.** OtterTune (Van Aken et al. 2017) reuses past workload observations with Gaussian-process models to tune database knobs, and CDB-Tune (Zhang et al. 2019) learns a deep reinforcement-learning policy from trial rewards; a survey (Herodotou,

Chen, and Lu 2020) covers rule-based, cost-model, experiment-driven, learned and adaptive tuners for data-processing systems. Bayesian optimization (Snoek, Larochelle, and Adams 2012) and services such as Vizier (Golovin et al. 2017) are general black-box baselines. For the Linux kernel, STUN (Lee, Jung, and Jo 2022) tunes scheduler parameters with Q-learning for static workloads. None of these methods must assume independent parameters: a surrogate can model interactions, and a graph-aware optimizer can receive exactly the evidence given to an LLM. A comparison that grants offline pairwise-search results only to the language-model system therefore cannot isolate language-model reasoning; our protocol includes both cold-start and shared-KB optimizers and accounts separately for offline and online costs.

**Language models for configuration.** DB-BERT (Trummer 2022) mines tuning hints from manuals and refines them with trial runs; GPTuner (Lao et al. 2024) uses GPT-extracted knowledge to prune knobs and ranges before coarse-to-fine Bayesian optimization; and  $\lambda$ -Tune (Giannakouris and Trummer 2025) generates complete configuration scripts and bounds the cost of evaluating candidates. For operating systems, AutoOS (Chen et al. 2024) uses an LLM-driven observe-prune-propose-act-correct loop over Linux kernel build options, and TuneAgent (Lin et al. 2025) trains an LLM agent with rule-based reinforcement learning over Kconfig option groups. These systems target build-time or database configuration and evaluate end-to-end performance. SemantOS instead targets runtime controls and focuses on the enforcement boundary: complete bundles are validated and gated by a component independent of the proposer, and the evaluation separates outcome labels from enforcement decisions. We do not claim to outperform these systems; no head-to-head comparison is reported.

**Grounding and uncertainty.** Retrieval-augmented generation (Lewis et al. 2020) conditions outputs on retrieved documents, and self-consistency (Wang et al. 2023) aggregates multiple sampled outputs. SemantOS uses both, but treats agreement among samples only as a heuristic score to be calibrated, not as a probability.

**Risk control.** Selective classification (Geifman and El-Yaniv 2017) abstains to meet a target risk. Conformal Risk Control (Angelopoulos et al. 2024) controls expected monotone losses, and its non-exchangeable extension (Farinhas et al. 2024), like weighted conformal prediction (Barber et al. 2023) and adaptive conformal inference (Gibbs and Candès 2021), addresses drifting data with explicit weights or on-line updates. None implies that an arbitrary sliding window restores a fixed risk level after a change detected by, e.g., ADWIN (Bifet and Gavalda 2007). Our rank gate is a restricted class-conditional result, not a replacement for these frameworks or a new drift guarantee.

**Explanation faithfulness.** Provenance lists what context was supplied, not necessarily what influenced a recommendation; faithfulness must be evaluated separately from plausibility (Jacovi and Goldberg 2020). ContextCite (Cohen-Wang et al. 2024) attributes generated output to context by

ablation. We adopt matched context-deletion interventions to test whether cited edges and traces influence the proposed bundle; operator preference alone cannot establish faithfulness.

**Kernel interfaces.** Linux documents runtime controls such as `vm.swappiness`, `vm.dirty_ratio` and `vm.dirty_background_ratio` (Linux kernel contributors 2024), and per-thread controls such as timer slack and CPU affinity (Linux man-pages project 2026a,b). Availability and semantics must be checked on the running kernel; the example policy uses a small allowlist of documented controls.

## Motivation and Testable Questions

SemantOS is motivated by three hypotheses. First, a configuration’s utility may depend on workload and hardware. Second, joint knob effects may be non-additive. Third, performance, failure probability and intervention cost may conflict. Each hypothesis requires measured interventions with explicit baselines.

For a loss  $L$ , define an interaction contrast from four independently measured configurations:

$$I_{ab} = L(a, b) - L(a, b_0) - L(a_0, b) + L(a_0, b_0).$$

For lower-is-better loss, negative  $I_{ab}$  indicates a beneficial non-additive interaction relative to the chosen baseline. A bootstrap interval must resample independent runs, not individual requests from a single run. The sign can change with workload, value range or kernel version, so an edge needs its scope, configuration values, measurement IDs and uncertainty interval. Such an edge describes an empirical interaction, not an unconditional causal coupling.

A `DEPENDS-ON` edge requires a validated admissibility rule or measured range conditioning; it cannot be inferred solely from the sign of  $I_{ab}$ . The released seed graph is empty until edges pass this test. The principal question is whether a reasoner can use independently acquired evidence more effectively than an optimizer with the same inputs, after charging the cost of acquiring that evidence.

## SemantOS Design

### Control Contract

The architecture contains telemetry, a graph and trace store, a reasoner, a runtime and an operator console. The REST prototype connects these components, but its runtime is a *dry run*: it stages an in-memory bundle and never writes a kernel setting, routes traffic or demonstrates a kernel rollback.

**Evidence and score.** The reasoner retrieves typed neighbors and trace text (Lewis et al. 2020), then requests  $k = 3$  model samples (Wang et al. 2023). For a knob-value pair proposed in  $v$  samples, its heuristic score is  $u = 0.7(1 - v/k) + 0.3(1 - w)$ , clipped to  $[0.02, 1]$ , where  $w$  is the strongest retrieved synergy weight (zero when absent). This is neither predictive entropy nor a calibrated probability. Backend failure must be recorded; repeated copies of a fallback do not constitute independent model votes. The implementation abstains when no configured model produces

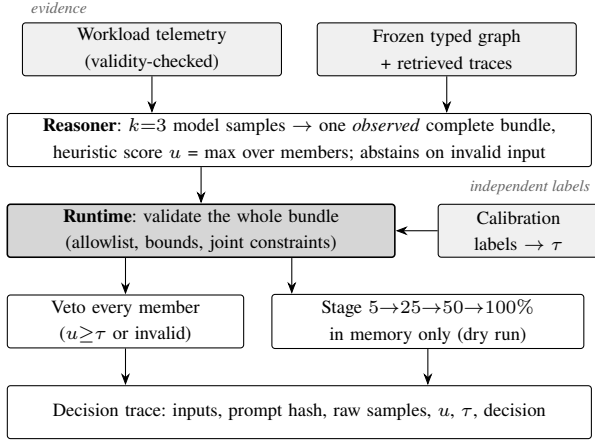


Figure 1: SemantOS decision path as implemented. Evidence, proposal and enforcement are separate components, so the same evidence can be given to a non-LLM optimizer and gate errors can be counted against independently defined labels. The REST runtime never writes kernel settings; a separate per-thread actuator (evaluated later) applies, verifies and restores timer slack and affinity in owned child processes.

recommendations. Supplied evidence is recorded as provenance, not certified as a faithful explanation.

The aggregator chooses a modal complete bundle actually present in a model response, rather than splicing independently winning knob values into an unseen combination. Backend failures retain their type and failed samples remain in the score denominator. Responses expose the complete supplied context, prompt hash and raw samples; exceeding the context budget causes abstention instead of truncating evidence. A model name in this record does not by itself establish immutable checkpoint provenance.

**Bundle contract.** A request contains a nonempty bundle identifier and a list of recommendations with `id`, `knob`, `proposed`, `uncertainty`, and optional `rationale` and `provenance`. Each request submits one bundle. Duplicate knobs, nonfinite scores, unsupported controls, and example-policy bound violations reject the entire request. Dirty-ratio changes must supply both values and satisfy the example policy that the background ratio is less than the foreground ratio. This is a declared policy, not proof that a configuration improves any workload. If any score satisfies  $u \geq \tau$ , all members are vetoed; otherwise all are staged together. A concurrent bundle receives a conflict response. Dry-run results return `applied=[]` and list simulated members separately.

**Deployment requirements.** A real actuator must snapshot all old values, validate the full target configuration and available host interfaces, serialize updates, verify readback, and restore every old value on failure. Multiple `sysctl` writes are not a kernel transaction: partial exposure must be measured and cannot be described as atomic hardware state. Canary shares require separate hosts or an appropriate isolation mechanism; changing a host-global `sysctl` cannot isolate

### Algorithm 1 Implemented dry-run bundle path

---

**Require:** Validated bundle  $B$ , fixed score function, calibration threshold  $\tau$

- 1: Validate every member and joint constraints before changing state
- 2: **if** another bundle is active **then**
- 3:   Return conflict
- 4: **end if**
- 5: **if**  $\max_{a \in B} u(a) \geq \tau$  **then**
- 6:   Record veto for all members; return
- 7: **end if**
- 8: Stage the complete bundle in memory at 5%
- 9: **for** requested stage in  $\{25, 50, 100\}\%$  **do**
- 10:   Advance the simulated bundle; record the transition
- 11: **end for**
- 12: On cancellation, clear the complete simulated bundle
- 13: Never infer an unsafe/safe training label from simulated completion

---

a fraction of requests on that host. Missing, stale or malformed end-to-end telemetry must stop promotion. These capabilities are requirements, not features demonstrated by the dry-run state machine.

**One illustrative decision trace.** Consider supplied context with example values 5 and 15 for the two dirty ratios, and retrieved evidence IDs  $e_1, t_1$ . Suppose all three model samples agree on this pair and  $w = 0$ , giving  $u = 0.30$ . If an independently obtained threshold is  $\tau = 0.25$ , the whole bundle is vetoed; if  $\tau = 0.35$ , it can enter the dry-run canary after joint validation. The audit record includes the values, scores, threshold and decision. These numbers illustrate the contract and are not a measured recommendation or calibrated operating point. The initial artifact threshold is zero until calibration data is explicitly supplied.

**Scheduling scope.** The reasoner awaits model samples synchronously, and we make no fast-path latency or resource-footprint claim. A future nonblocking deployment path should separate synchronous policy enforcement from asynchronous proposals, record the objective weights and policy version per decision, and refresh graph and calibration snapshots with versioned evidence. No *delay* state or immutable audit storage is claimed: logs are append-only files within a process, without tamper resistance.

### Probability Statements for Gated Acceptance

Let  $Y = 1$  denote an independently defined unsafe outcome and  $A = 1$  acceptance. An outcome label is not defined by whether a rollback happened. For example, a benchmark can predeclare a request deadline, count observed failures or deadline exceedances, and define an action-level breach from these raw observations in a fixed window. Rejected actions have unknown counterfactual outcomes unless evaluated in an isolated experiment; they must not automatically be labeled safe or unsafe.

**Theorem 1** (Cost identity). *For costs  $c_{FN}, c_{FP}, \lambda \geq 0$ , define  $C = c_{FN} \mathbf{1}\{Y = 1, A = 1\} + c_{FP} \mathbf{1}\{Y = 0, A =$*

$0\} + \lambda D$ , where  $D \geq 0$  is measured exposure or SLO debt. Write  $\pi = \Pr(Y = 1)$ ,  $q = \Pr(A = 1 \mid Y = 1)$  and  $r = \Pr(A = 0 \mid Y = 0)$ . Then

$$\mathbb{E}C = c_{FN}\pi q + c_{FP}(1 - \pi)r + \lambda \mathbb{E}D.$$

For a zero-probability class its associated product is zero.

*Proof.* Expand each indicator expectation as its joint probability and use the definition of conditional probability. Linearity requires no independence.  $\square$

This is an accounting identity, not a claim of novelty or a safety bound. A bound  $q \leq \alpha$  controls misses among unsafe actions. A different bound  $\Pr(Y = 1 \mid A = 1) \leq \alpha$  instead gives  $\Pr(Y = 1, A = 1) \leq \alpha \Pr(A = 1)$ , not  $\pi\alpha$ . Likewise  $r$  is a veto rate among safe actions, not the proportion of vetoed actions that are safe. We report all denominators separately.

**Proposition 1** (Restricted rank gate). *Fix a score function before calibration. Suppose  $n$  unsafe calibration scores and the next unsafe test score are jointly exchangeable. Set  $j = \lfloor \alpha(n+1) \rfloor$  for  $0 < \alpha < 1$ . If  $j = 0$ , veto all; otherwise let  $\tau$  be the  $j$ th smallest unsafe calibration score, and accept only when  $u < \tau$ . Then the unsafe-class acceptance probability, marginal over calibration and test scores, is at most  $j/(n+1) \leq \alpha$ .*

*Proof.* With distinct scores, the test score has a uniform rank among  $n+1$  exchangeable scores. It lies strictly below the  $j$ th calibration order statistic only in the first  $j$  ranks. Ties with strict acceptance can only reduce this probability;  $j=0$  accepts nothing.  $\square$

For bundle-level calibration,  $u$  denotes the maximum member score and  $Y$  the independently observed bundle outcome. Per-knob labels alone do not establish a bundle-level guarantee.

Selecting a smaller threshold by a cost rule cannot enlarge the acceptance event. Selecting a larger threshold can invalidate the bound; therefore the implementation uses the minimum of the rank threshold and optional cost-selected cap. No unsafe calibration examples implies veto-all. Calibration-window diagnostics are not test-set estimates.

**Drift and selective observation.** Sliding windows and a drift detector (Bifet and Gavalda 2007) do not guarantee exchangeability or a deterministic recovery time. Under an explicitly justified total-variation bound on the *joint calibration-test law*, the probability of the same event changes by at most that bound; a marginal score-distance statement alone does not establish it. We neither estimate that distance nor claim recovery. Online labels selected by acceptance introduce additional selection bias. The runtime holds its threshold at zero on detected change until an explicit refresh; refresh remains an exploratory operation, not proof of safety.

## Measurements and Evaluation Protocol

### New Local Measurements

We executed three bounded microbenchmarks on the available host: an Intel Core i5-3570 with four logical CPUs, approximately 31.2 GiB RAM, and Linux 6.17.0-23-generic.

Table 1: Measured local baseline. Mean of run medians, and mean of run P95 values with 95% Student- $t$  CI half-width across 10 runs. Last column is the percentage of errors or operations exceeding a predeclared 50 ms deadline. No tuning gain is implied.

| Workload      | Median ms | P95 ms            | Events % |
|---------------|-----------|-------------------|----------|
| file_fsync    | 1.903     | $2.035 \pm 0.043$ | 0.00     |
| sqlite_commit | 1.870     | $2.187 \pm 0.273$ | 0.00     |
| zlib_compress | 1.474     | $2.038 \pm 0.356$ | 0.00     |

These runs characterize the measurement pipeline: no kernel knob was modified and no LLM or controller was compared.

Each workload had 10 separate invocations of 100 measured operations, preceded by 10 warm-up operations. Workload-run order was shuffled with seed 4088. SQLite inserts a 4 KiB value and commits each transaction in WAL mode with synchronous=FULL. The file test overwrites a 64 KiB region and calls fsync. The compression test compresses a fixed seeded 64 KiB payload at zlib level 6 and verifies decompression. Latency is wall-clock operation duration from a monotonic nanosecond clock; the compression measurement includes verification. The raw log retains every operation, run ID, deadline and error flag, with a SHA-256 hash and software/environment manifest.

The anomaly denominator is all measured operations, including failures; an operation counts once if either it fails or exceeds the deadline. This label is independent of veto and rollback. Zero events in these short runs is not evidence of zero population risk. Run-level intervals describe variability on this shared host and do not account for uncontrolled interference, warm-cache bias or correlated system state. The closed-loop load does not characterize queueing under an externally fixed arrival rate. We do not pool heterogeneous workload latencies or infer kernel tuning effects from these baseline timings.

### Measured Per-Process Kernel Interventions

We additionally executed real Linux timer-slack and CPU-affinity interventions in fresh owned child processes, without modifying host-wide sysctls. The two factors were timer slack (50  $\mu$ s or 1 ms, set by PR\_SET\_TIMERSLACK) and affinity (all four available CPUs or CPU 0). Every child checked kernel readback before recording events. This isolated measurement driver is separate from the dry-run Se-mantOS actuator.

For each periodic wake-up interval (2 or 5 ms), five training blocks and ten subsequent test blocks evaluated all four configurations. Within each block, the eight interval/configuration cases were randomly ordered using seed 4088. Each fresh process performed ten warm-up and 80 measured wake-ups against absolute monotonic-clock targets. The plan was written before execution; the configuration minimizing mean training-run P95 lateness was frozen before test execution. This is temporal holdout within one host and workload family, not the independent workload/hardware split required for claims of generalization. The complete 120-

Table 2: Held-out kernel intervention measurements. P95 is the mean of ten run-level P95 wake-up latenesses. Misses count lateness exceeding the predeclared 200  $\mu$ s deadline. CPU affinity is all four CPUs or one CPU.

| Period | Slack/CPU | P95 ( $\mu$ s) | Misses  |
|--------|-----------|----------------|---------|
| 2      | 50us/all  | 89.5           | 2/800   |
| 2      | 50us/one  | 109.4          | 18/800  |
| 2      | 1ms/all   | 1016.8         | 800/800 |
| 2      | 1ms/one   | 968.9          | 771/800 |
| 5      | 50us/all  | 94.2           | 2/800   |
| 5      | 50us/one  | 133.8          | 22/800  |
| 5      | 1ms/all   | 1022.6         | 663/800 |
| 5      | 1ms/one   | 1073.7         | 680/800 |

process experiment retained 9,600 event timestamps, read-backs, frozen selections and hashes. No external competing load was introduced.

The training selector chose the explicit control (50  $\mu$ s slack, all CPUs) at both intervals: there is no selected-policy improvement over that control. Its test P95 was  $89.5 \pm 9.2$  and  $94.2 \pm 6.9 \mu$ s at 2 and 5 ms, respectively (95% Student-*t* half-width across ten runs). Raising slack to 1 ms with all CPUs increased P95 by  $927.3 \pm 16.4$  and  $928.4 \pm 10.0 \mu$ s in paired blocks. Affinity-only differences were  $19.9 \pm 36.7$  and  $39.6 \pm 62.6 \mu$ s; these intervals include zero. These exploratory intervals are unadjusted for multiple comparisons and shared-host temporal dependence. The slack effect is consistent with timer coalescing; no power measurement, graph discovery, LLM advantage or staged rollback is inferred. A miss is defined by observed wake-up time, independently of any safety gate.

## Required Controlled Comparisons

**Matched information and budgets.** To test the LLM’s incremental value, compare (1) the unchanged kernel baseline, (2) a frozen expert profile, (3) a conventional optimizer without the KB, (4) the same optimizer with identical graph and retrieved traces, (5) a deterministic graph policy, and (6) SemantOS. Fix the allowed knob values, workload arrivals, hardware allocation, trial and wall-clock budgets, safety policy and objective weights. Record model inference, graph construction, retrieval, calibration and human setup costs separately. Report both online and total costs; a cold-start comparison alone cannot isolate semantic reasoning.

Ablations remove graph context, retrieval, or the LLM while retaining the other inputs and the runtime. A no-runtime comparison belongs only in an isolated testbed. The KB may carry all useful information; the shared-KB optimizer is the direct test of that alternative. Distinct random-search and optimizer implementations must not be merged into a generic BO/RL result.

**Leakage prevention.** Before graph induction, define immutable groups by workload family, hardware class and acquisition session. Maintain disjoint train, retrieval, calibration and test roles; no held-out group may contribute to LLM training, pairwise sweeps, retrieval traces or threshold se-

lection. Freeze hashes of graph snapshots, source measurements, prompts and calibrators before opening test results. Temporal reuse must be forward-only. The artifact includes a manifest validator to reject group overlap and unknown lineage.

**Outcome definitions and safety counts.** For each proposed bundle record proposed, malformed, constraint-rejected, score-vetoed, accepted, staged, completed and rolled-back counts, together with exposure duration and affected requests. Count bundles and individual knobs separately. For independently labeled actions report TP, FP, FN and TN for the *gate*; separately report rollout rollback counts and causes. Give both *#unsafe accepted*/*#unsafe labeled* and *#unsafe accepted*/*#accepted labeled*. Unknown labels remain unknown. Rollback precision is undefined without independently labeled executed actions; veto precision is not its substitute.

A lower threshold accepts a subset of the same fixed candidate list, but the unsafe fraction among survivors need not be monotone. Report absolute unsafe counts, accepted counts, request-level deadline misses and exposure under each threshold on the same frozen candidates.

**Aggregation and trade-offs.** Use paired seeds and runs and report absolute per-workload and per-host metrics before percentage reductions against the explicitly named unchanged-kernel baseline. Aggregate normalized ratios, not raw milliseconds across heterogeneous tasks. Bootstrap independent runs or workload-host groups as appropriate; state the unit of resampling. A Pareto plot requires multiple measured operating points per method across a common budget or objective-weight sweep. Connect only genuinely nondominated measured points and report uncertainty; a dashed line through one point per method is not evidence of a frontier.

**Drift and faithfulness.** A drift experiment must retain the score function version,  $\alpha$ , window size, number of unsafe calibration examples, change schedule, threshold trajectory and held-out outcomes at each decision. Evaluate on subsequent data, not on the window used to select the threshold. Report observed misses, denominators and uncertainty; any claimed recovery deadline needs its own assumptions and evidence. For explanation faithfulness, delete cited edges/traces and matched uncited controls while fixing model seeds and objective weights, then measure bundle changes and attribution errors. Record the complete original and perturbed prompts. No human study is needed to begin this intervention test.

## Prospective Role-Separated Follow-up

To exercise the protocol’s role separation, frozen selection and independent labels, we collected a separate prospective dataset using the same four timer-slack/affinity configurations. The plan was saved before acquisition. Training, retrieval, calibration and test used periods of 2, 3, 4 and {5, 8} ms, respectively. Each period had ten randomized four-configuration blocks, with five warm-ups and 40 recorded events per fresh child. This produces 200 measurement processes and 8,000 events. Roles are period-disjoint, not workload-family- or hardware-independent: all runs use

Table 3: Follow-up selector replay: mean of ten test-run P95 values. Misses are observed lateness above  $200\ \mu\text{s}$ . Shared rows across selectors are intentional: selectors choosing the same configuration use the same held-out observations.

| Period | Selector      | P95 ( $\mu\text{s}$ ) | Misses  |
|--------|---------------|-----------------------|---------|
| 5      | control       | 179.7                 | 13/400  |
| 5      | empirical min | 179.7                 | 13/400  |
| 5      | graph min     | 179.7                 | 13/400  |
| 5      | seeded random | 811.6                 | 248/400 |
| 8      | control       | 155.7                 | 13/400  |
| 8      | empirical min | 155.7                 | 13/400  |
| 8      | graph min     | 155.7                 | 13/400  |
| 8      | seeded random | 642.2                 | 225/400 |

the same periodic workload on one shared host. The acquisition manifest validates declared groups, run hashes and graph/calibrator lineage; the analyzer additionally checks freeze timestamps, record completeness and kernel read-backs.

**Shared-information non-LLM comparison.** The empirical selector minimizes mean training-run P95 over the four configurations. A saturated two-factor representation encodes the same four means as a baseline, two main effects and their interaction; minimizing it adds no information to table lookup. Both selectors have the same 40 training runs and zero online search trials. An unchanged-control selector and a seeded uniform-random selector provide references. Choices are frozen before calibration and test. We evaluate these fixed selectors by paired *offline replay* on the same measured test grid, not by separate closed-loop executions. A block is the uncertainty unit, and paired reductions are calculated within each period against its explicit control.

Both informed selectors chose  $50\ \mu\text{s}$  slack and all CPUs, identical to control; their paired improvement is exactly zero in this replay. Control P95 is  $179.7 \pm 102.0$  and  $155.7 \pm 64.4\ \mu\text{s}$  at 5 and 8 ms (95% Student- $t$  half-width across ten blocks). The large intervals illustrate shared-host variability. The training interaction contrast is  $-18.6\ \mu\text{s}$ ; the prespecified normalized-contrast bootstrap emitted no edge because its interval included zero. We do not turn a nonsignificant contrast into a dependency rule or a semantic-reasoning gain. Acquisition took 6.54 s for training, 8.29 s for retrieval and 10.15 s for calibration; graph construction and selection took 0.10 s, included in calibration-phase wall time. These are measured local costs, not a comparison against LLM inference costs. Retrieval records are retained for the model audit but are not needed by the table selector.

**Independent labels and gate misses.** For this experiment only, the score is a configuration’s training deadline-miss fraction, fixed before calibration; it is not the REST reasoner’s self-consistency score. A run is labeled unsafe when more than 10% of its 40 observed events exceed the predeclared  $200\ \mu\text{s}$  deadline. Thus the label depends on measured timing, independently of a veto or rollback. Every candidate is executed in an isolated owned process, permitting gate replay with observed outcomes for accepted and vetoed

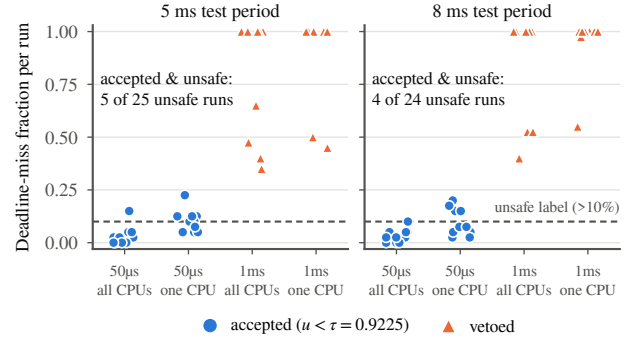


Figure 2: Frozen-gate replay on the 80 held-out test runs; each point is one run. Positive means unsafe (dashed line) and a gate-positive decision is veto. Counts at every tested  $\alpha$  (all give  $\tau = .9225$ ): 5 ms, 20 accepted / 20 vetoed, TP 20, FP 0, FN 5, TN 15; 8 ms, 20 / 20, TP 20, FP 0, FN 4, TN 16. Blue points above the line are the accepted unsafe runs. This is a held-out diagnostic, not prevented production harm. Generated from raw logs by `plot_gate_replay.py`.

candidates. Calibration uses only 4 ms runs. The three pre-specified  $\alpha$  values  $\{.05, .10, .20\}$  all yield  $\tau = .9225$  because the unsafe-score order statistics are tied.

Figure 2 shows every test run. Unsafe acceptance is  $5/25 = 20.0\%$  and  $4/24 = 16.7\%$  among unsafe runs; unsafe fractions among accepted runs are  $5/20 = 25.0\%$  and  $4/20 = 20.0\%$ . Veto precision is 20/20 in each period, which is not rollback precision. The observed miss rates exceed the nominal 5% and 10% levels. Calibration and test periods differ, runs share a host, and the proposition is marginal under exchangeability; these results neither verify its deployment assumptions nor establish recovery. The gate is frozen throughout test, with no retrospective adjustment. The two test periods are different operating conditions, not a drift-recovery trajectory. All counts, thresholds and per-run scores are retained rather than replacing unfavorable outcomes with calibrated claims.

**Actual staged application and restoration.** A separate adapter snapshots the child thread’s timer slack and affinity, validates the target, applies both settings, verifies read-back, and restores both originals in a *finally* path, including partial-application failures. These per-thread controls are documented Linux interfaces (Linux man-pages project 2026a,b). Updates are sequential, not an atomic kernel transaction. The driver observes stages of 8, 16 and 32 events and stops when a stage’s deadline-miss fraction exceeds 10%. Stages are observation counts, not fractions of production traffic. All four configurations are exercised in ten blocks at each test period, independently of the gate replay.

Across 80 fresh executions, 66 stop on a stage breach and 14 complete all stages; all 80 restore and read back both original values. At 5 and 8 ms, observed deadline misses are respectively 186/816 and 190/880; summed exposure durations are 4.106 and 7.066 s. Early stops reduce the number of observed events, so these fractions cannot be com-

pared directly with fixed-duration, no-stop runs to claim an anomaly reduction. The stop criterion itself is not independent ground truth for rollback precision. The adapter demonstrates bounded actuation and restoration in an owned child; it is not wired into the REST VM-sysctl runtime and does not establish traffic isolation, recovery from irreversible effects or production safety.

**Model and explanation audit.** The executable audit supplies only frozen training and retrieval evidence, records exact prompts, responses, model manifest, seeds, decoding limits and elapsed time, and checks every cited identifier. It compares full context, no graph, no retrieval and model-only inputs. For a valid full-context response it deletes cited context and same-kind, equal-count uncited controls using the same seed; absent matched controls are reported as non-estimable. This tests action sensitivity, not operator preference. A local Llama 3.2 manifest was recorded for an attempted run; the inference endpoint failed with a process-permission error in the measurement environment before any inference, so no valid model response was obtained. The stored failure is evidence of an unexecuted experiment, not a zero LLM effect or a faithfulness result. These two research questions remain unresolved.

## Discussion and Limitations

The central idea is the separation of structured evidence, proposals, and enforcement. The artifact demonstrates a dry-run REST decision contract, a separate bounded per-thread actuator, raw measurements and non-LLM selector replay. Measured graph induction on the new training sweep emits no edge. It lacks a deployed VM-sysctl bundle actuator, traffic isolation, a verified trained checkpoint, and controlled end-to-end LLM comparative results. Prompting a model to obey graph edges does not enforce those edges; independent validation is required. The current REST runtime enforces only its explicit example constraints.

The available telemetry prototype contains scheduler and block-I/O demonstration proxies, including timestamp-derived buckets; those are not end-to-end request latencies. They must not calibrate an empirical SLO claim. Actual workload drivers must record completion, failure and deadline data. The new local benchmarks do so for their narrow tasks but do not substitute for web serving, TPC-C, Kafka/Spark, GPU inference or audio pipelines.

The rank gate’s assumptions can fail under drift, selective labeling, or a score model updated using calibration data. A canary can suffer harm before detection, and rollback cannot undo every side effect. No no-worse-than-default, catastrophic-regression prevention, Pareto superiority or universal safety claim follows from the prototype. These limitations define concrete measurements and engineering prerequisites for the next evaluation.

## Ethical and Responsible Deployment

The measurements run local software workloads and involve no human participants or personal data. The REST runtime performs no kernel writes; the experimental adapter

changes only its owned child thread’s timer slack and affinity, restores both settings, and performs no host-wide sysctl writes. Broader interventions require an isolated testbed, documented rollback limits and appropriate authorization. Any future human study requires prospective institutional review and informed consent.

AI coding assistants were used in developing parts of the code, prose and analysis. Every reported number is produced by released scripts from raw logs whose SHA-256 hashes are recorded, so that it can be recomputed independently; no generated value substitutes for an unavailable observation.

## Conclusion

SemantOS is a research prototype for inspectable, knowledge-grounded kernel-tuning proposals with independent bundle gating. We contribute a bundle-level control contract, correctly conditioned risk statements, and a role-separated evaluation protocol with raw-log verification. Applied to real per-thread kernel interventions, the protocol shows that informed non-LLM selectors gain nothing over the default, that a frozen calibrated threshold can miss more unsafe runs than its nominal level under a period shift, and that bounded staged actuation restored the original settings in every execution. Whether an LLM improves decisions over an equally informed optimizer, and whether staging reduces independently measured harm in real deployments, are the next questions the released tools are built to answer.

## References

- Angelopoulos, A. N.; Bates, S.; Fisch, A.; Lei, L.; and Schuster, T. 2024. Conformal Risk Control. In *International Conference on Learning Representations*.
- Barber, R. F.; Candès, E. J.; Ramdas, A.; and Tibshirani, R. J. 2023. Conformal Prediction beyond Exchangeability. *The Annals of Statistics*, 51(2): 816–845.
- Bifet, A.; and Gavaldà, R. 2007. Learning from Time-Changing Data with Adaptive Windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining*, 443–448.
- Chen, H.; Wen, Y.; Cheng, L.; Kuang, S.; et al. 2024. AutoOS: Make Your OS More Powerful by Exploiting Large Language Models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, 7511–7525.
- Cohen-Wang, B.; Shah, H.; Georgiev, K.; and Mądry, A. 2024. ContextCite: Attributing Model Generation to Context. In *Advances in Neural Information Processing Systems*, volume 37.
- Farinhas, A.; Zerva, C.; Ulmer, D.; and Martins, A. F. T. 2024. Non-Exchangeable Conformal Risk Control. In *International Conference on Learning Representations*.
- Geifman, Y.; and El-Yaniv, R. 2017. Selective Classification for Deep Neural Networks. In *Advances in Neural Information Processing Systems*, volume 30, 4878–4887.
- Giannakouris, V.; and Trummer, I. 2025.  $\lambda$ -Tune: Harnessing Large Language Models for Automated Database System

- Tuning. *Proceedings of the ACM on Management of Data*, 3(1).
- Gibbs, I.; and Candès, E. 2021. Adaptive Conformal Inference Under Distribution Shift. In *Advances in Neural Information Processing Systems*, volume 34.
- Golovin, D.; Solnik, B.; Moitra, S.; Kochanski, G.; Karro, J.; and Sculley, D. 2017. Google Vizier: A Service for Black-Box Optimization. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1487–1495.
- Herodotou, H.; Chen, Y.; and Lu, J. 2020. A Survey on Automatic Parameter Tuning for Big Data Processing Systems. *ACM Computing Surveys*, 53(2): 43:1–43:37.
- Jacovi, A.; and Goldberg, Y. 2020. Towards Faithfully Interpretable NLP Systems: How Should We Define and Evaluate Faithfulness? In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 4198–4205.
- Lao, J.; Wang, Y.; Li, Y.; Wang, J.; Zhang, Y.; Cheng, Z.; Chen, W.; Tang, M.; and Wang, J. 2024. GPTuner: A Manual-Reading Database Tuning System via GPT-Guided Bayesian Optimization. *Proceedings of the VLDB Endowment*, 17(8): 1939–1952.
- Lee, H.; Jung, S.; and Jo, H. 2022. STUN: Reinforcement-Learning-Based Optimization of Kernel Scheduler Parameters for Static Workload Performance. *Applied Sciences*, 12(14): 7072.
- Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, volume 33, 9459–9474.
- Lin, H.; Li, Y.; Luo, H.; et al. 2025. TuneAgent (first released as OS-R1): Agentic Operating System Kernel Tuning with Reinforcement Learning. arXiv:2508.12551.
- Linux kernel contributors. 2024. Documentation for /proc/sys/vm/.
- Linux man-pages project. 2026a. PR\_SET\_TIMERSLACK(2const).
- Linux man-pages project. 2026b. sched\_setaffinity(2).
- Snoek, J.; Larochelle, H.; and Adams, R. P. 2012. Practical Bayesian Optimization of Machine Learning Algorithms. In *Advances in Neural Information Processing Systems*, volume 25, 2951–2959.
- Trummer, I. 2022. DB-BERT: A Database Tuning Tool that “Reads the Manual”. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)*.
- Van Aken, D.; Pavlo, A.; Gordon, G. J.; and Zhang, B. 2017. Automatic Database Management System Tuning Through Large-scale Machine Learning. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*, 1009–1024.
- Wang, X.; Wei, J.; Schuurmans, D.; Le, Q. V.; Chi, E. H.; Narang, S.; Chowdhery, A.; and Zhou, D. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations*.
- Zhang, J.; Liu, Y.; Zhou, K.; Li, G.; et al. 2019. An End-to-End Automatic Cloud Database Tuning System Using Deep Reinforcement Learning. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, 415–432.